

# Chapter 12 — CAP, sharding, and consistency

Distribution introduces trade-offs among consistency, availability, and partition tolerance

# Learning objectives

- By the end of this part

# The CAP theorem

- Partition tolerance simultaneously
- Consistency means every read returns the most recent write
- Availability means every request receives a response even if some nodes are down
- Partition tolerance means the system continues when the network splits nodes apart

# CP versus AP under partition

- Because partitions are unavoidable in real networks
- Under partition
- The choice depends on whether stale reads or temporary unavailability is more costly for

# Sharding and Example 12.10

- Example 12.10 — hands-on module
- Data sharding partitions data across servers to improve scalability, fault tolerance
- Example 12.10 shows a common sharding key choice
- Explore the chapter example module
- View files: ``modules/chapter12/example10/``

## Example 12.10 — listing

```
"""Example 12.10 – shard customer keys by geography or customer ID."""

from __future__ import annotations

import hashlib
from collections import Counter

def shard_for_customer(customer_id: str, n_shards: int) -> int:
    """Return a stable shard index for a customer ID."""
    digest = hashlib.md5(customer_id.encode("utf-8")).hexdigest()
    return int(digest, 16) % n_shards

def main() -> None:
    """Show ID hashing and region mapping as two common sharding
    keys."""
```

## Example 12.11 — Strong Consistency in Financial Ledgers

- Example 12.11 — hands-on module
- Strong consistency guarantees that any read returns the most recent write regardless of
- Example 12.11 requires that guarantee for balances
- Explore the chapter example module
- View files: ``modules/chapter12/example11/``

## Example 12.13 — Eventual Consistency in DynamoDB and Cassandra

- Example 12.13 — hands-on module
- Writes propagate to other nodes, but nodes may temporarily disagree
- Example 12.13 lists stores that favor availability under partition
- Amazon DynamoDB and Apache Cassandra commonly emphasize availability and partition
- Explore the chapter example module
- View files: ``modules/chapter12/example13/``



# Choosing a consistency model

- For many web and social applications
- E-commerce platforms, content delivery networks
- Conversely, sensitive financial or health-related data often require strong consistency to
- Design hinges on application needs and the consistency, availability

# Takeaways

- CAP says partitions force a choice between consistency and availability in practice
- Sharding spreads keys; replication preserves availability after failures
- Strong consistency fits ledgers and similar integrity-critical work
- Match the model to whether stale reads or downtime hurts more

# Next

- Complete the quiz for this part
- The next part shifts from storage mechanics to dataset versioning at platform scale